PROJECT FINAL REPORT

Project Title: Large-Scale Customer Sentiment Analysis using the Yelp Open Dataset

Name: Rishitha Adapa

Primary Paper:

Title: Thumbs Up? Sentiment Classification Using Machine Learning Techniques

Authors: Bo Pang, Lillian Lee, Shivakumar Vaithyanathan

Year: 2002

Conference: EMNLP

Link: https://www.cs.cornell.edu/home/llee/papers/sentiment.pdf

1. Introduction

1.1 Problem Statement

In today's digital era, platforms such as Yelp generate a massive amount of customer reviews in natural language. These reviews contain valuable insights about customer satisfaction, product quality, and service experience. However, manually analyzing such large volumes of text data is extremely time-consuming and inefficient.

The main problem addressed in this project is to automatically classify Yelp reviews into positive or negative sentiment categories using machine learning techniques. This task is a classic Natural Language Processing (NLP) problem known as sentiment analysis, where the goal is to understand the emotional tone behind textual data.

1.2 Motivation

The motivation behind this project is to automate sentiment analysis of customer reviews, which helps businesses make better decisions. Manual analysis is time-consuming and inefficient, especially for large datasets.

There are also several challenges involved in this task. Text data is unstructured and noisy, and reviews vary in writing style and vocabulary. Handling such data effectively is one of the key difficulties in sentiment analysis.

1.3 Open Questions in the Domain

While working on this problem, several key questions arise:

How can raw text be converted into numerical features for machine learning models?

Which machine learning model performs best for sentiment classification?

How much preprocessing is required for good performance?

Can simple models perform well without using deep learning?

How do models handle edge cases such as mixed sentiment reviews?

These questions guide the design and evaluation of the project.

1.4 A Brief Overview of the Approach to Address the Challenges

In order to address the challenges of analyzing large-scale, unstructured text data, this project follows a structured machine learning approach. The first step involves text preprocessing, where the raw review data is cleaned by removing unnecessary characters, converting text to lowercase, and eliminating common stopwords while preserving important words such as “not” that influence sentiment.

After preprocessing, the cleaned text is converted into a numerical format using the Bag-of-Words (BoW) model. This method represents each review as a vector of word frequencies, allowing machine learning models to process textual data effectively.

Next, multiple supervised learning models, including Naive Bayes, Logistic Regression, and Support Vector Machine (SVM), are trained on the processed data. These models are then evaluated using standard performance metrics such as accuracy to determine their effectiveness.

Finally, the results are analyzed to identify the best-performing model and understand its strengths and limitations. This overall approach helps in efficiently handling noisy text data and provides a scalable solution for sentiment classification.

2. Backgrounds

2.1 Summary of other related researches

Sentiment analysis has been extensively studied in the field of Natural Language Processing, and several research works have demonstrated the effectiveness of machine learning techniques for text classification.

The primary research work followed in this project is:

Bo Pang, Lillian Lee, Shivakumar Vaithyanathan (2002)

“Thumbs Up? Sentiment Classification Using Machine Learning Techniques”

Presented at: EMNLP (Empirical Methods in Natural Language Processing)

In this paper, the authors introduced sentiment analysis as a document classification problem. They used the Bag-of-Words representation, where each document is represented as a vector of word frequencies. The study evaluated machine learning models such as Naive Bayes and Logistic Regression, and showed that these models can achieve strong performance even with simple feature representations.

Another important research work is:

Thorsten Joachims (1998)

“Text Categorization with Support Vector Machines: Learning with Many Relevant Features”

Published in: European Conference on Machine Learning (ECML)

This work demonstrated that Support Vector Machines (SVM) are highly effective for text classification tasks. The study showed that SVM performs particularly well in high-dimensional feature spaces, which is a key characteristic of text data.

Additionally, the following work provides further insights:

Armand Joulin, Edouard Grave, Piotr Bojanowski, Tomas Mikolov (2016)

“Bag of Tricks for Efficient Text Classification”

Published in: European Chapter of the Association for Computational Linguistics (EACL)

This paper focused on efficient text classification methods and showed that simple models combined with effective feature representation techniques

can achieve strong performance. It emphasized that computational efficiency can be maintained without sacrificing accuracy.

2.2 Pros and Cons

The approaches discussed above have several advantages as well as some limitations.

Pros

These methods are simple and easy to implement

They require relatively low computational resources

They perform well even on large-scale text datasets

Training and prediction are fast

Cons

They have limited ability to understand deeper meaning or context

They struggle with sarcasm or complex sentence structures

Their performance depends heavily on the quality of feature representation

2.3 How the other work is related to the main method

This project is directly based on the approach proposed by Bo Pang, Lillian Lee, and Shivakumar Vaithyanathan (2002). In their work, sentiment analysis is treated as a classification problem, and the same idea is followed in this project.

The use of the Bag-of-Words model for feature extraction is also inspired by their methodology. Similarly, the machine learning models used in this project, such as Naive Bayes and Logistic Regression, are derived from the same approach proposed in their study.

The inclusion of Support Vector Machine (SVM) in this project is influenced by the work of Thorsten Joachims (1998), which demonstrated that SVM is highly

effective for text classification tasks, especially when dealing with high-dimensional data.

Additionally, the idea that simple models can achieve strong performance is supported by the work of Armand Joulin, Edouard Grave, Piotr Bojanowski, and Tomas Mikolov (2016), which emphasizes efficient text classification using simple feature representations.

Overall, this project builds upon these research works and applies their ideas to the Yelp dataset, demonstrating that classical machine learning methods can still achieve strong performance in sentiment analysis.

3. Methods

3.1 How I applied the algorithms and methods in this project

In this project, I used a machine learning-based approach to classify Yelp reviews as either positive or negative. Since machine learning models cannot directly understand text data, the first step was to convert the raw text into a numerical format.

To achieve this, I used the Bag-of-Words (BoW) technique. In this method, each review is converted into a vector based on word frequencies. Each word becomes a feature, and the value represents how often that word appears in the review. To make the model more efficient and avoid handling a very large number of features, I limited the vocabulary to the top 5000 most frequent words.

Creating the Bag of Words model

In [40]:

```
# CountVectorizer converts text into a matrix of token counts
from sklearn.feature_extraction.text import CountVectorizer

# max_features limits the vocabulary size to the most frequent 5000 words
# This helps reduce memory usage and improves model efficiency
cv = CountVectorizer(max_features=5000)

# Fit the model on the training data and transform it into a Bag-of-Words matrix
# Each row represents a review and each column represents a word in the vocabulary
X_train_bow = cv.fit_transform(x_train).toarray()

# Transform the test data using the same vocabulary learned from the training data
# This ensures consistency between training and testing datasets
X_test_bow = cv.transform(x_test).toarray()
```

After converting the text into numerical features, I trained three different machine learning models on the dataset:

Naive Bayes

Naive Bayes is a probabilistic model that works based on Bayes' theorem. It assumes that all features are independent of each other. Even though this assumption is not always true, it performs well in text classification tasks because of its simplicity and speed.

Logistic Regression

Logistic Regression is a linear model that predicts the probability of a class label. In this project, it learns how different words contribute to the sentiment of a review and uses that information to classify new data.

Support Vector Machine (SVM)

Support Vector Machine is a classification model that tries to find the best boundary between classes. It works well for text data because it can handle high-dimensional features effectively.

- All three models were trained using the same dataset and feature representation so that their performance could be compared fairly.

3.2 How I structured the overall implementation

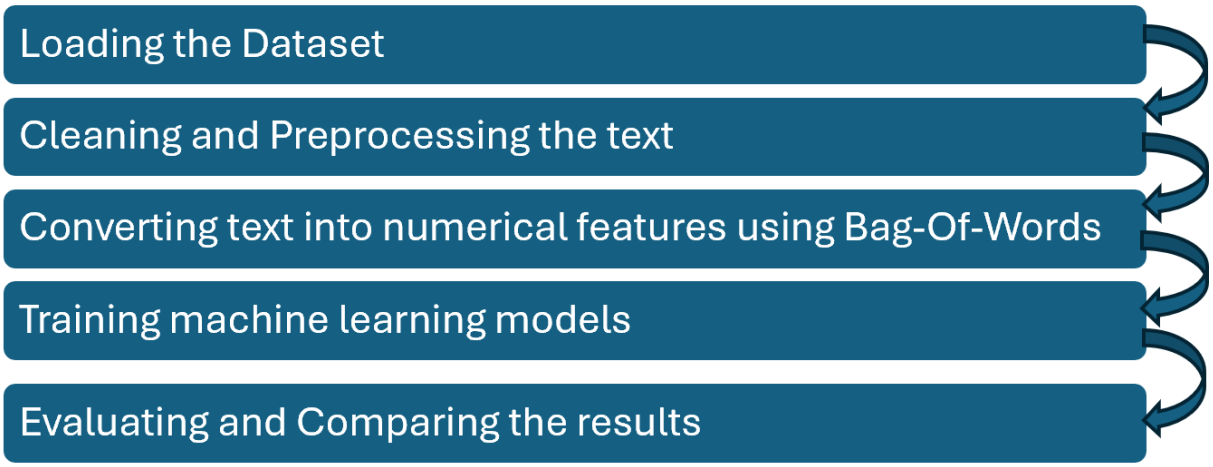
To solve this problem, I followed a clear and structured workflow. The process starts with loading the dataset, where I imported the Yelp review data required for sentiment analysis. After that, I performed cleaning and preprocessing of the text, which included converting all text to lowercase, removing special characters, and eliminating unnecessary words.

Once the data was cleaned, I converted the text into numerical features using the Bag-of-Words technique. This step is important because machine learning models cannot directly understand text, so the data must be transformed into a numerical format.

After feature extraction, I trained multiple machine learning models, including Naive Bayes, Logistic Regression, and Support Vector Machine (SVM). These models were trained on the same dataset to ensure a fair comparison.

Finally, I evaluated and compared the results of all models to determine which model performs best for sentiment classification.

As shown in the below figure, this workflow represents the complete pipeline followed in this project, starting from data loading and ending with model evaluation.



3.3 How I handled data preprocessing

Data preprocessing was an important step in this project because raw text data contains a lot of noise and inconsistencies.

I applied the following preprocessing steps:

Converted all text to lowercase for consistency

Removed special characters, numbers, and punctuation

Split the text into individual words

Removed common stopwords such as “the”, “is”, and “and”

Kept the word “not” because it plays an important role in sentiment

These steps helped in cleaning the data and improving its quality. As a result, the machine learning models were able to focus on meaningful words and produce better predictions.

```python
import re
import nltk
nltk.download('stopwords')

from nltk.corpus import stopwords

# Loading English stopwords
stop_words = set(stopwords.words('english'))

# Keep "not" because it is important for sentiment analysis
stop_words.discard('not')


# Function to clean and remove stopwords
def clean_text(text):

    # Convert text to lowercase
    text = text.lower()

    # Remove numbers and special characters
    text = re.sub('[^a-zA-Z]', ' ', text)

    # Split the text into words
    words = text.split()

    # Remove stopwords
    words = [word for word in words if word not in stop_words]

    # Join the cleaned words back into a sentence
    text = ' '.join(words)

    return text


# Apply cleaning to the training data
x_train = x_train.apply(clean_text)

# Apply the same cleaning to the test data
x_test = x_test.apply(clean_text)
```

```
[nltk_data] Downloading package stopwords to
```

```
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\adapa\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

In [30]:

```
x_train.head()
```

Out[30]:

```
0    unfortunately frustration dr goldberg patient ...
1    going dr goldberg years think one st patients ...
2    know dr goldberg like moving arizona let tell ...
3    writing review give heads see doctor office st...
4    food great best thing wings wings simply fanta...
Name: text, dtype: object
```

In [31]:

```
x_test.head()
```

Out[31]:

```
0    contrary reviews zero complaints service price...
1    last summer appointment get new tires wait sup...
2    friendly staff starbucks fair get anywhere els...
3    food good unfortunately service hit miss main ...
4    even car filene basement worth bus trip waterf...
Name: text, dtype: object
```

4. Experiments

4.1 Testing the models and discussing the results

In this project, I trained three machine learning models: Naive Bayes, Logistic Regression, and Support Vector Machine (SVM) on the Yelp review dataset. After preprocessing the text data and converting it into numerical features using the Bag-of-Words approach, all models were trained using the same dataset to ensure a fair comparison.

The performance of the models was evaluated using accuracy. The results obtained are:

Naive Bayes → 86% accuracy

Logistic Regression → 92% accuracy

Support Vector Machine → 91% accuracy

Evaluating model performance using accuracy score and classification report

In [45]:

```
# Evaluating Model Performance

from sklearn.metrics import accuracy_score, classification_report

# Accuracy for each model
nb_acc = accuracy_score(y_test, y_pred_nb)
lr_acc = accuracy_score(y_test, y_pred_lr)
svm_acc = accuracy_score(y_test, y_pred_svm)

print("Naive Bayes Accuracy:", nb_acc)
print("Logistic Regression Accuracy:", lr_acc)
print("SVM Accuracy:", svm_acc)

# Detailed classification metrics
print("\nNaive Bayes Report:\n", classification_report(y_test, y_pred_nb))
print("\nLogistic Regression Report:\n", classification_report(y_test, y_pred_lr))
print("\nSVM Report:\n", classification_report(y_test, y_pred_svm))
```

```
Naive Bayes Accuracy: 0.8576052631578948
Logistic Regression Accuracy: 0.9172894736842105
SVM Accuracy: 0.913921052631579


Naive Bayes Report:
               precision    recall  f1-score   support

           1       0.89      0.81      0.85     19000
           2       0.83      0.90      0.86     19000

    accuracy                           0.86     38000
   macro avg       0.86      0.86      0.86     38000
weighted avg       0.86      0.86      0.86     38000


Logistic Regression Report:
               precision    recall  f1-score   support

           1       0.92      0.92      0.92     19000
           2       0.92      0.92      0.92     19000

    accuracy                           0.92     38000
   macro avg       0.92      0.92      0.92     38000
weighted avg       0.92      0.92      0.92     38000


SVM Report:
               precision    recall  f1-score   support

           1       0.91      0.92      0.91     19000
           2       0.91      0.91      0.91     19000

    accuracy                           0.91     38000
   macro avg       0.91      0.91      0.91     38000
weighted avg       0.91      0.91      0.91     38000
```

From these results, it can be observed that Logistic Regression performs the best among the three models. SVM also shows strong performance, while Naive Bayes performs slightly lower but is faster.

4.2 Reproducing the paper's experiments

The approach used in this project is inspired by the work of Bo Pang, Lillian Lee, and Shivakumar Vaithyanathan (2002), where sentiment analysis is treated as a classification problem using machine learning techniques.

I was able to follow a similar approach by using Bag-of-Words for feature extraction and applying machine learning models such as Naive Bayes and Logistic Regression. The results obtained in this project are consistent with the general findings of the paper.

However, the results are not exactly identical. This is mainly because:

The dataset used in this project (Yelp reviews) is different from the dataset used in the original paper

The size and nature of the data vary

The preprocessing steps and implementation details may differ

4.3 Additional experiments and observations

In this project, I mainly focused on comparing different machine learning models on the Yelp dataset. Although I did not use additional datasets, I analyzed how different models behave on the same data.

Some key observations are:

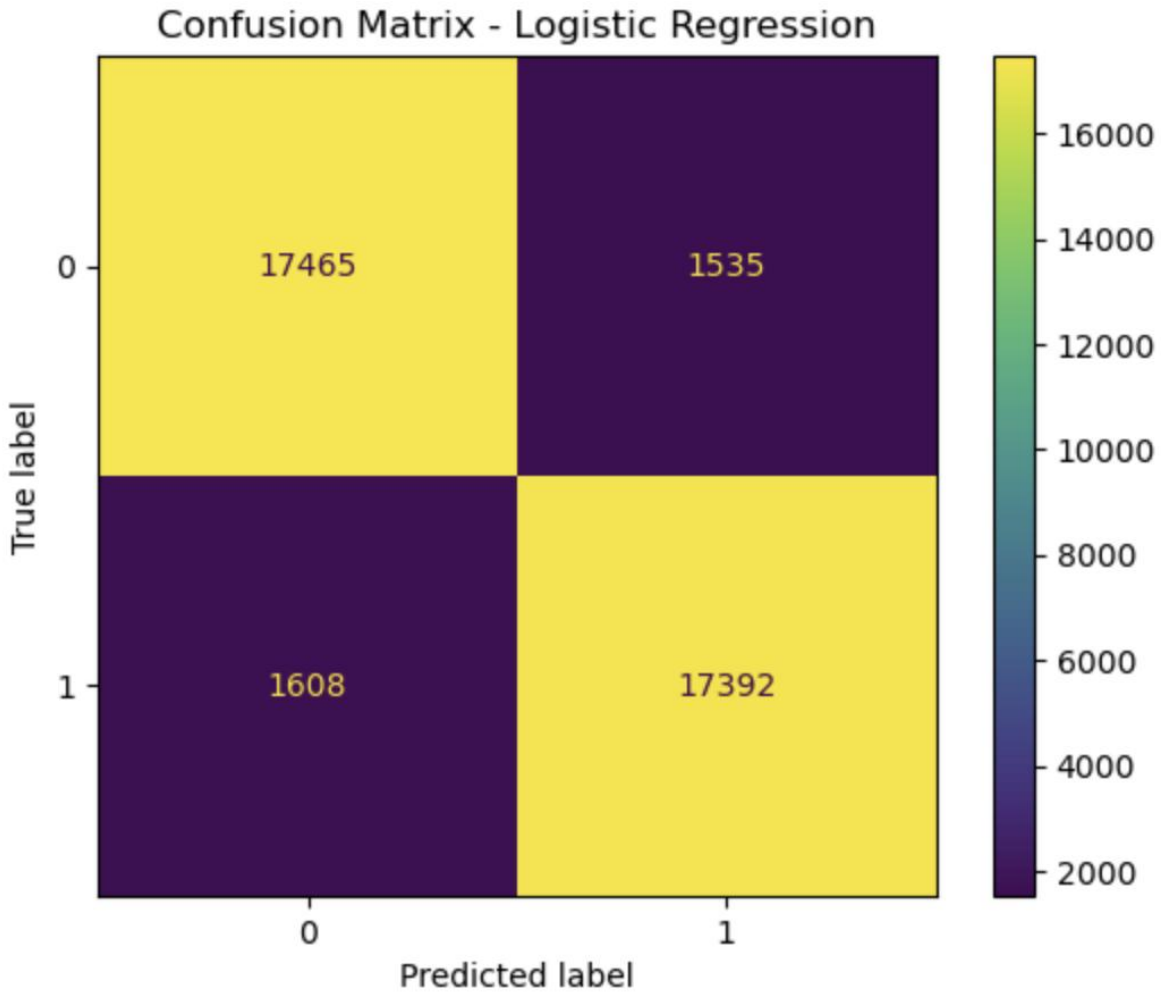
Logistic Regression gives the highest accuracy

SVM performs very closely to Logistic Regression

Naive Bayes is faster but less accurate

Most errors occur in reviews where the sentiment is unclear or mixed

To further analyze the performance of the best model, I used a confusion matrix for Logistic Regression.



The confusion matrix shows that the model correctly classifies most of the reviews, with only a small number of misclassifications. It also indicates that the model performs well for both positive and negative classes, demonstrating balanced performance.

4.4 My thoughts on the results and overall solution

From this project, I learned that simple machine learning models can perform very well for sentiment analysis when combined with proper preprocessing and feature extraction.

The Bag-of-Words approach proved to be effective in converting text into numerical features. Logistic Regression achieved the best performance, showing that it is a suitable model for this type of problem.

Overall, the solution developed in this project is efficient and produces good results. It can be further improved by using advanced techniques such as TF-IDF, deep learning models, or improved preprocessing methods.

5. Conclusions

This project successfully built a sentiment classification system using Yelp reviews.

Logistic Regression achieved the best performance with around 92% accuracy.

The project demonstrates that simple machine learning models combined with basic preprocessing techniques can effectively solve sentiment analysis problems.

Future Work:

Explore TF-IDF representation

Use deep learning models (LSTM, BERT)

Perform hyperparameter tuning

Improve preprocessing techniques

6. Sharing Agreement

I agree to share my work as an example for next semester.

I am comfortable sharing my name along with the work.

7. My Contributions

In this project, I independently implemented a complete sentiment analysis pipeline using the Yelp Review dataset. I performed data preprocessing to clean and standardize the text data, including removing noise, handling stopwords, and preserving important words such as “not” for sentiment understanding.

I applied the Bag-of-Words technique to convert text into numerical features and trained multiple machine learning models, including Naive Bayes, Logistic Regression, and Support Vector Machine. I evaluated the performance of these models using accuracy and compared their results to identify the best-performing model.

I also analyzed the performance of the best-performing model, Logistic Regression, using a confusion matrix. This helped me understand how well the model classified positive and negative reviews and identify any misclassifications. Based on this analysis, I was able to interpret the strengths and limitations of the model.

While the overall methodology was inspired by existing research papers, the implementation, experimentation, and analysis were carried out by me.

8. References

1. Bo Pang, Lillian Lee, and Shivakumar Vaithyanathan (2002).

Thumbs Up? Sentiment Classification Using Machine Learning Techniques.

Proceedings of the 2002 Conference on Empirical Methods in Natural Language Processing (EMNLP).

Link: https://aclanthology.org/W02-1011/

2. Thorsten Joachims (1998).

Text Categorization with Support Vector Machines: Learning with Many Relevant Features.

European Conference on Machine Learning (ECML).

Link: https://link.springer.com/chapter/10.1007/BFb0026683

3. Armand Joulin, Edouard Grave, Piotr Bojanowski, and Tomas Mikolov (2016).

Bag of Tricks for Efficient Text Classification.

Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics (EACL).

Link: https://arxiv.org/pdf/1607.01759

4. Yelp Dataset (Kaggle).

Yelp Dataset for Business Reviews and Sentiment Analysis.

Link: https://www.kaggle.com/datasets/yelp-dataset/yelp-dataset

5. Scikit-learn Documentation.

Machine Learning Library for Python.

Link: https://scikit-learn.org/stable/

6. Natural Language Toolkit (NLTK) Documentation.

Python Library for Text Processing.

Link: https://www.nltk.org/